



CSC220 · WEB DEVELOPMENT II · WEEK 2 · SESSION 2

async/await & the Fetch API

Working with JSON and public APIs

Stamford International University — Information Technology / Computer Science

Lecturers: Thinzar Aung Win · Zakariyya Bature

Today's agenda

30'

async / await

Cleaner promise syntax, error handling with try/catch

30'

JSON & the Fetch API

Requesting data, checking responses, parsing JSON, rendering to the DOM

35'

Class activity

Build a mini app that consumes a public API — 10 points

10'

Wrap-up

Common pitfalls, homework, preview of Week 3 (Git & GitHub)

Promises that read like normal code

```
then-style.js

// yesterday: .then chain
login(user)
  .then(token => getProfile(token))
  .then(profile => getPosts(profile.id))
  .then(posts => render(posts));
```

```
await-style.js

// today: same logic with async/await
const show = async user => {
  const token = await login(user);
  const profile = await getProfile(token);
  const posts = await getPosts(profile.id);
  render(posts);
};
```

- `async` before a function makes it always return a promise
- `await` pauses inside that function until the promise settles, then gives you the value
- `await` only works inside an `async` function (or at module top level)
- It's the same promises underneath — just cleaner to read and debug

Why it matters: nearly all Express and MongoDB code from Week 4 onward is written with `async/await`.

Error handling with try / catch

```
tryCatch.js

const loadPosts = async id => {
  try {
    const profile = await getProfile(id);
    const posts   = await getPosts(profile.id);
    return posts;
  } catch (err) {
    console.error("Failed:", err.message);
    return [];           // safe fallback
  } finally {
    hideSpinner();       // always runs
  }
};
```

- try wraps the awaited steps; catch receives any rejection from any of them
- Same roles as .then/.catch/.finally — familiar shape from synchronous code
- Return a sensible fallback or rethrow — don't swallow errors silently
- An unhandled rejection prints a scary console error: always catch in real apps

PART 2

JSON & the Fetch API

Requesting live data from servers — the browser side of everything you'll build with Express and MongoDB.



The data format of the web

student.json

```
{
  "name": "Beam",
  "age": 20,
  "skills": ["HTML", "CSS", "JS"],
  "address": { "city": "Bangkok" },
  "active": true
}
```

json.js

```
// text -> object
const obj = JSON.parse(text);
obj.skills[2];           // "JS"

// object -> text (to send/store)
const text2 = JSON.stringify(obj);
```

- JSON = JavaScript Object Notation: text that looks like JS objects
- Rules: double quotes around keys and strings; no functions, no comments, no trailing commas
- Every API this trimester speaks JSON — requests and responses
- MongoDB documents (Week 7) look almost identical to JSON

fetch: two awaits to your data

```
fetch.js

const url =
  "https://jsonplaceholder.typicode.com/users";

const loadUsers = async () => {
  const res = await fetch(url);    // 1: headers

  if (!res.ok) {                  // 404? 500?
    throw new Error(`HTTP ${res.status}`);
  }

  const data = await res.json();  // 2: body
  console.log(data);             // array!
};
```

- Await #1 — `fetch(url)` resolves when response headers arrive
- Check `res.ok` / `res.status`: a 404 is NOT a rejection — `fetch` only rejects on network failure
- Await #2 — `res.json()` parses the body (it's a promise too)
- Default is a GET request; POST with options comes in Week 8 when we build our own API

Rendering API data (Week 1 + Week 2)

```

const list = document.querySelector("#users");

const showUsers = async () => {
  list.innerHTML = "<li>Loading...</li>";
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(res.status);
    const users = await res.json();

    list.innerHTML = users
      .map(({ name, email }) =>
        `<li>${name} - ${email}</li>`)
      .join("");
  } catch (err) {
    list.innerHTML = "<li>Could not load</li>";
  }
};

showUsers();

```

- 1 **Loading state**
show feedback immediately
- 2 **Fetch + check**
await, verify res.ok
- 3 **Transform**
map + destructuring + template literals
- 4 **Render or fail**
innerHTML from data, friendly error

This loading → data → error pattern is exactly what React's `useEffect` automates in Week 11.

Free public APIs to practice with

JSONPlaceholder

`jsonplaceholder.typicode.com`

Fake users, posts, todos — perfect first API

REST Countries

`restcountries.com`

Country data: names, capitals, flags, population

Open-Meteo

`open-meteo.com`

Weather forecasts by latitude/longitude, no key needed

Bored API

`bored-api.appbrewery.com`

Random activity suggestions — one small object

PokeAPI

`pokeapi.co`

Pokémon data — large nested JSON to explore

Activity 4: API mini-app (35 min · 10 points)

Build a small page that fetches and displays live data.

Requirements

- 1. A button starts the fetch (addEventListener from Week 1)
- 2. Use async/await with try/catch, and check res.ok
- 3. Show a loading message while waiting
- 4. Render at least 5 items (or 5 fields) with map + template literals
- 5. Show a friendly error message if the request fails

Suggested ideas

- User directory from JSONPlaceholder (names + emails)
- Country explorer: capital + population from REST Countries
- “I’m bored” button → random activity from the Bored API
- Pokédex card: name, types and stats from PokeAPI
- Stretch: a second button, a search input, or Promise.all over two endpoints

Submission: index.html + app.js to LMS “Week 2 – Activity 4” before the end of class. **10 points · during class time only · no late submission.**

Common async pitfalls



Forgetting await

you get a pending Promise object instead of data — console shows Promise {<pending>}



Skipping the res.ok check

a 404 still 'succeeds' — then res.json() fails on an error page



await outside an async function

SyntaxError — wrap the code in an async function first



Opening the page as file://

fetch is blocked — always use Live Server for API work



Awaiting in a loop unnecessarily

independent requests should use Promise.all, not one-by-one awaits

You can now work with live data

- 1 Write async functions and await promise results
- 2 Handle failures with try / catch / finally
- 3 Read and produce JSON with parse / stringify
- 4 Fetch from an API: two awaits and a res.ok check
- 5 Render live API data to the DOM with loading and error states

Homework

Assignment 1 opens this week (see LMS). Reading: MDN “Fetching data from the server”; explore one new public API.

Next week — Git & GitHub

Version control: repositories, commits, branching and collaboration. Create a free github.com account before class.